



Оснoвы ASP NET Core

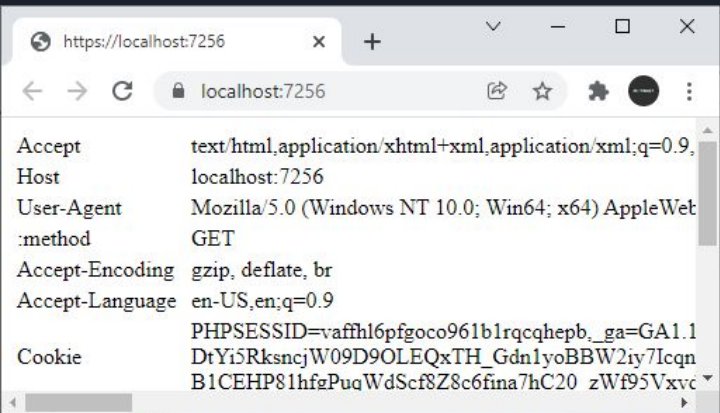


HttpRequest. Получение данных запроса

Свойство **Request** объекта `HttpContext` представляет объект `HttpRequest` и хранит информацию о запросе в виде следующих свойств:

- **Body**: предоставляет тело запроса в виде объекта **Stream**
- **BodyReader**: возвращает объект типа **PipeReader** для чтения тела запроса
- **ContentLength**: получает или устанавливает заголовок `Content-Length`
- **ContentType**: получает или устанавливает заголовок `Content-Type`
- **Cookies**: возвращает коллекцию куки (объект `Cookies`), ассоциированных с данным запросом
- **Form**: получает или устанавливает тело запроса в виде форм
- **HasFormContentType**: проверяет наличие заголовка `Content-Type`
- **Headers**: возвращает заголовки запроса
- **Host**: получает или устанавливает заголовок `Host`
- **HttpContext**: возвращает связанный с данным запросом объект **HttpContext**
- **IsHttps**: возвращает `true`, если применяется протокол `https`
- **Method**: получает или устанавливает метод HTTP
- **Path**: получает или устанавливает путь запроса в виде объекта `RequestPath`
- **PathBase**: получает или устанавливает базовый путь запроса. Такой путь не должен содержать завершающий слеш
- **Protocol**: получает или устанавливает протокол, например, HTTP
- **Query**: возвращает коллекцию параметров из строки запроса
- **QueryString**: получает или устанавливает строку запроса
- **RouteValues**: получает данные маршрута для текущего запроса
- **Scheme**: получает или устанавливает схему запроса HTTP

Для получения заголовков применяется свойство **Headers**, которое представляет тип **IHeaderDictionary**. Например, получим все заголовки запроса и выведем их на веб-страницу:





HttpRequest. Получение данных запроса

Для большинства стандартных заголовков HTTP в этом интерфейсе определены одноименные свойства, например, для заголовка "content-type" определено свойство **ContentType**, а для заголовка "accept" - свойство **Accept**:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async(context) =>
{
    var acceptHeaderValue = context.Request.Headers.Accept;
    await context.Response.WriteAsync($"Accept: {acceptHeaderValue}");
});

app.Run();
```

Также подобные заголовки, а также какие-то кастомные заголовки, для которых не определены подобные свойства, можно получить как и любой дугой элемент словаря:

```
var acceptHeaderValue = context.Request.Headers["accept"];
```

Для ряда заголовков в классе HttpRequest определены отдельные свойства: Host, Method, ContentType, ContentLength.

HttpRequest. Получение пути запроса

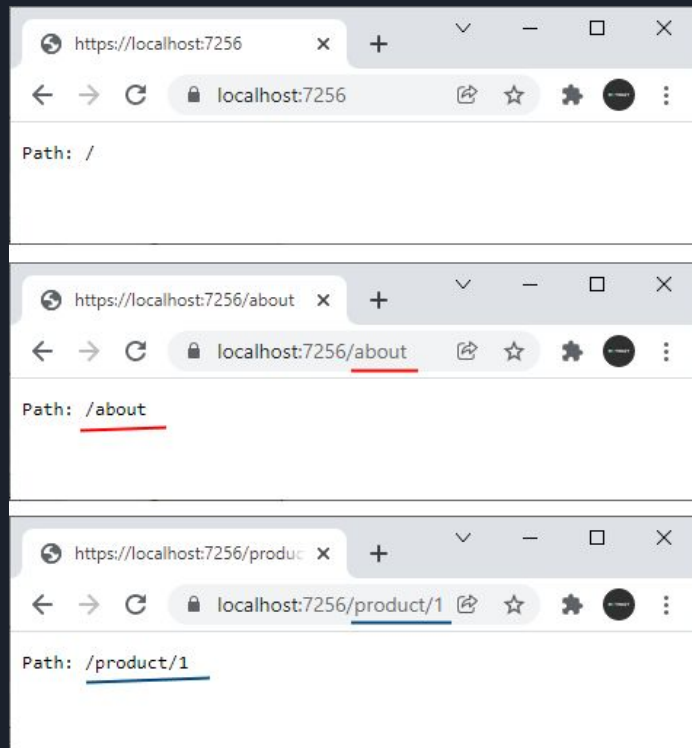
Свойство **path** позволяет получить запрошенный путь, то есть адрес, к которому обращается клиент:

```
var builder = WebApplication.CreateBuilder();
```

```
var app = builder.Build();
```

```
app.Run(async(context) => await context.Response.WriteAsync($"Path: {context.Request.Path}"));
```

```
app.Run();
```





HttpRequest. Получение пути запроса

Это свойство позволяет нам узнать, по какому адресу обращается пользователь. Например, мы можем определить условную обработку запроса в зависимости от запрошенного адреса:

Свойство **path** позволяет получить запрошенный путь, то есть адрес, к которому обращается клиент:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async(context) =>
{
    var path = context.Request.Path;
    var now = DateTime.Now;
    var response = context.Response;
    if (path=="/date")
        await response.WriteAsync($"Date: {now.ToShortDateString()}");
    else if (path == "/time")
        await response.WriteAsync($"Time: {now.ToShortTimeString()}");
    else
        await response.WriteAsync("Hello");
});
app.Run();
```

HttpRequest. Строка запроса

Свойство **QueryString** позволяет получить строку запроса. Строка запроса представляет ту часть запрошенного адреса, которая идет после символа **?** и представляет набор параметров, разделенных символом амперсанда **&**:

?параметр1=значение1&параметр2=значение2&параметр3=значение3

Каждому параметру с помощью знака равно передается некоторое значение. Стоит отметить, что строка запроса (query string) НЕ входит в путь запроса (path):

```
var builder = WebApplication.CreateBuilder();
```

```
var app = builder.Build();
```

```
app.Run(async (context) =>
```

```
{
```

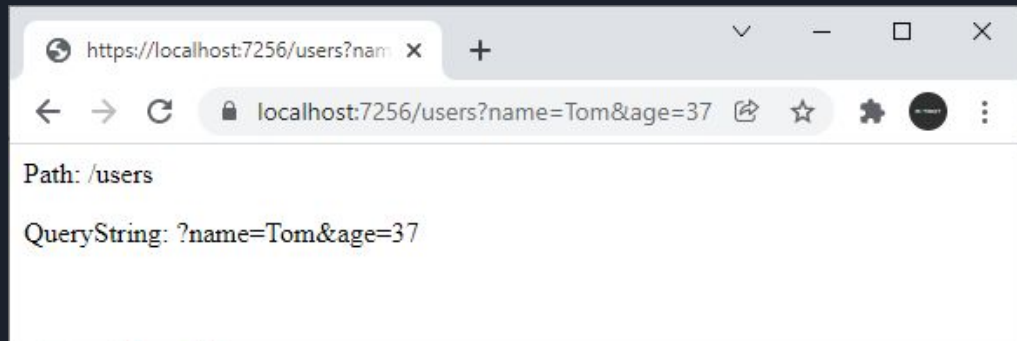
```
    context.Response.ContentType = "text/html; charset=utf-8";
```

```
    await context.Response.WriteAsync($"<p>Path: {context.Request.Path}</p>" +
```

```
        $"<p>QueryString: {context.Request.QueryString}</p>");
```

```
});
```

```
app.Run();
```



HttpRequest. Строка запроса

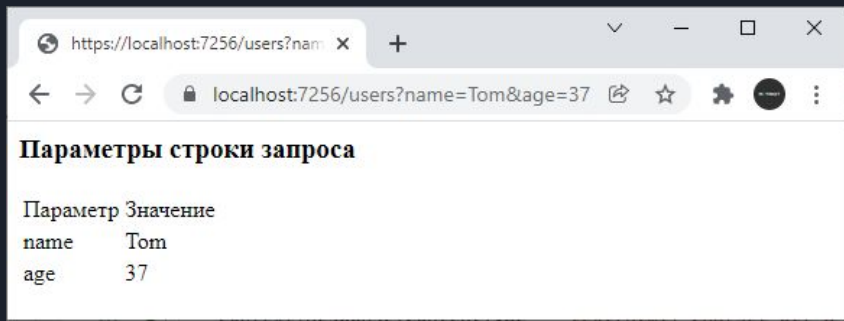
То есть в данном случае через строку запроса передаются два параметра. Первый параметр - name имеет значение "Tom". Второй параметр - age имеет значение 37.

С помощью свойства **Query** можно получить все параметры строки запроса в виде словаря:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async (context) =>
{
    context.Response.ContentType = "text/html; charset=utf-8";
    var stringBuilder = new System.Text.StringBuilder("<h3>Параметры строки запроса</h3><table>");
    stringBuilder.Append("<tr><td>Параметр</td><td>Значение</td></tr>");
    foreach (var param in context.Request.Query)
    {
        stringBuilder.Append($"<tr><td>{param.Key}</td><td>{param.Value}</td></tr>");
    }
    stringBuilder.Append("</table>");
    await context.Response.WriteAsync(stringBuilder.ToString());
});

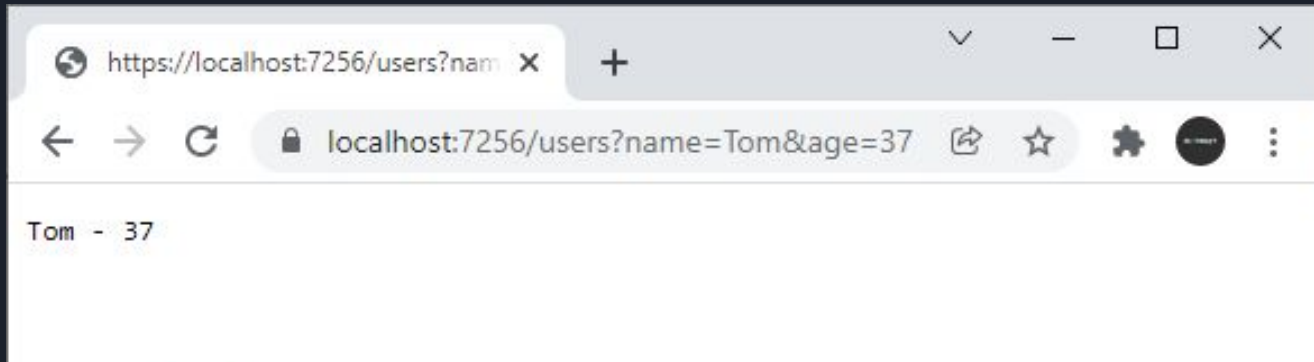
app.Run();
```



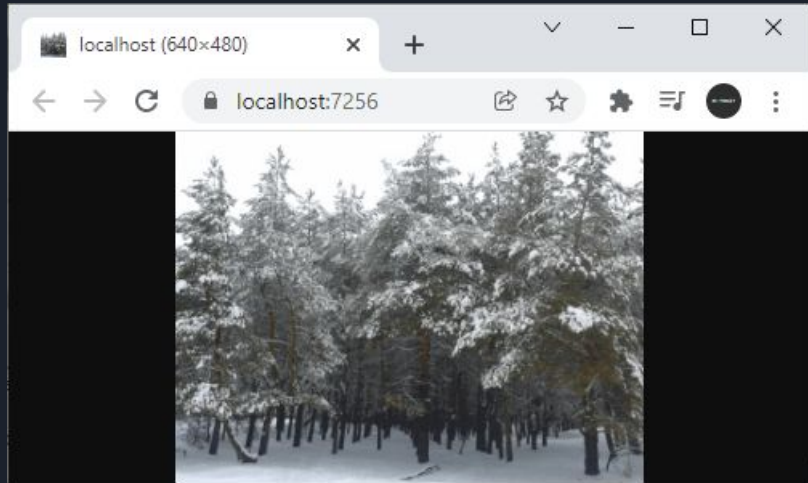
HttpRequest. Строка запроса

Соответственно можно вытащить из словаря Query значения отдельных параметров:

```
var builder = WebApplication.CreateBuilder();  
  
var app = builder.Build();  
  
app.Run(async context =>  
{  
    string name = context.Request.Query["name"];  
    string age = context.Request.Query["age"];  
    await context.Response.WriteAsync($"{name} - {age}");  
});  
  
app.Run();
```



Отправка файла



Для отправки файлов применяется метод **SendFileAsync()**, который получает либо путь к файлу в виде строки, либо информацию о файле в виде объекта **IFileInfo**. Например, допустим нам надо отправить файл по адресу "D:\\forest.jpg":

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
app.Run(async (context) => await context.Response.SendFileAsync("D:\\forest.jpg"));  
app.Run();
```

По умолчанию браузер попытается открыть файл. Так, в случае с изображениями они отображаются в браузере



Отправка файла

Также мы можем использовать относительные пути. Например, добавим в проект какой-нибудь файл (в моем случае это файл forest.jpg):

Для этого файла в окне свойств установим для опции **Copy to Output Directory** значение Copy if newer или Copy always, чтобы файл автоматически копировался в выходной каталог при построении приложения. И установим относительный путь относительно корня приложения:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Run(async(context) => await context.Response.SendFileAsync("forest.jpg"));  
  
app.Run();
```

Отправка html-страницы

Подобным образом мы можно отправлять и другие типы файлов, например, html-страницу. Так, определим в проекте новую папку, которую назовем **html**. В эту папку добавим новый файл **index.html**:

Определим в файле **index.html** следующий код:

```
<!DOCTYPE html>
<html>

  <head>
    <meta charset="utf-8" />
    <title>Test</title>
  </head>
  <body>
    <h2>Hello ASP.NET Core!</h2>
  </body>
</html>
```

Определим для отправки веб-страницы следующий код:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async(context) =>
{
    context.Response.ContentType = "text/html; charset=utf-8";
    await context.Response.SendFileAsync("html/index.html");
});

app.Run();
```

Отправка html-страницы

Теперь немного усложним задачу. Добавим в проект в папку **html** еще пару файлов. Назовем их, к примеру, **about.html** и **contact.html**.

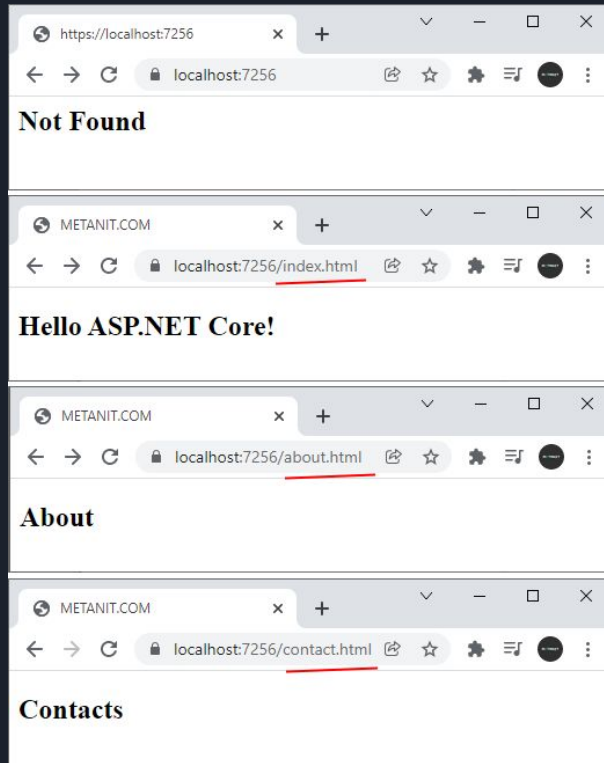
Для отправки этих файлов определим следующий код:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async (context) =>
{
    var path = context.Request.Path;
    var fullPath = $"html/{path}";
    var response = context.Response;

    response.ContentType = "text/html; charset=utf-8";
    if (File.Exists(fullPath))
    {
        await response.SendFileAsync(fullPath);
    }
    else
    {
        response.StatusCode = 404;
        await response.WriteAsync("<h2>Not Found</h2>");
    }
});

app.Run();
```



Стоит отметить, что в ASP.NET Core уже имеется встроенный middleware, который позволяет упростить работу со статическими файлами.

Загрузка файла

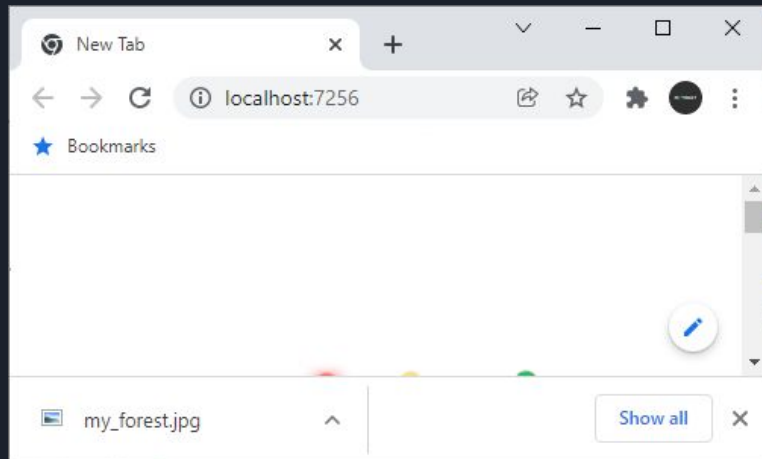
По умолчанию браузер пытается открыть отправляемый файл, что может быть полезно в случае файлов html - мы можем определить файл html и таким образом отправить клиенту веб-страницу. Но также может быть необходимо, чтобы браузер загружал файл без его открытия. В этом случае мы можем установить для заголовка "Content-Disposition" значение "attachment":

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async (context) =>
{
    context.Response.Headers.ContentDisposition = "attachment; filename=my_forest.jpg";
    await context.Response.SendFileAsync("forest.jpg");
});

app.Run();
```

В этом случае загруженный файл получит имя "my_forest.jpg"





FileInfo

В примерах выше применялась версия метода **SendFileAsync()**, которая получает путь к файлу в виде строки. Также можно использовать другую версию, которая получает информацию о файле в виде объекта **FileInfo**:

```
using Microsoft.Extensions.FileProviders;

var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async (context) =>
{
    var fileProvider = new PhysicalFileProvider(Directory.GetCurrentDirectory());
    var fileInfo = fileProvider.GetFileInfo("forest.jpg");

    context.Response.Headers.ContentDisposition = "attachment; filename=my_forest2.jpg";
    await context.Response.SendFileAsync(fileInfo);
});

app.Run();
```

В этом случае сначала необходимо определить объект **PhysicalFileProvider**, конструктор которого получает каталог для поиска файлов. В его метод `fileProvider.GetFileInfo()` передается путь к файлу в рамках этого каталога. А результатом метода является объект **FileInfo**, который передается в `SendFileAsync()`



Переадресация

Для выполнения переадресации у объекта **HttpResponse** определен метод **Redirect()**:

```
void Redirect(string location)
void Redirect(string location, bool permanent)
```

Первая версия выполняет временную переадресацию. В качестве параметра получает адрес для редиректа, а клиенту посылается статусный код 302.

Вторая версия метода также в качестве второго параметра получает булево значение, которое указывает, будет ли переадресация постоянной. Если этот параметр равен true, то переадресация будет постоянной, и в этом случае посылается статусный код 301. Если равен false, то переадресация временная, и посылается статусный код 302.

Переадресация

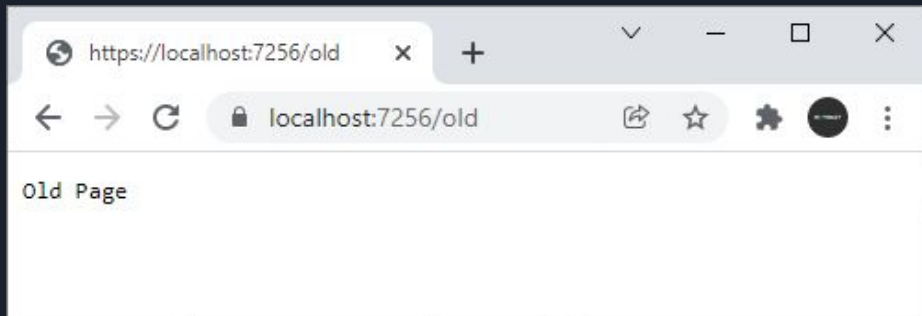
Допустим, у нас было следующее приложение

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async (context) =>
{
    if (context.Request.Path == "/old")
    {
        await context.Response.WriteAsync("Old Page");
    }
    else
    {
        await context.Response.WriteAsync("Main Page");
    }
});

app.Run();
```

При обращении по адресу "/old" приложение посылает сообщение "Old Page".



Переадресация

Теперь при обращении по адресу "/old" произойдет перенаправление на адрес "/new". В данном случае применяется редирект на локальный адрес в рамках приложения. Но также можно использовать редирект на внешние ресурсы:

```
if (context.Request.Path ==  
    "/old")  
{  
    context.Response.Redirect("https://ya.ru/search?q=8xbyte  
    .dev");  
}
```

Но затем мы решили сделать переадресацию с адреса "/old" на "/new". Используем для этого первую версию метода Redirect:

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Run(async (context) =>  
{  
    if (context.Request.Path == "/old")  
    {  
        context.Response.Redirect("/new");  
    }  
    else if (context.Request.Path == "/new")  
    {  
        await context.Response.WriteAsync("New Page");  
    }  
    else  
    {  
        await context.Response.WriteAsync("Main Page");  
    }  
});  
  
app.Run();
```



Создание простейшего API (без Swagger как было показано на практике)

Постараемся сами руками воссоздать API, без библиотек и вспомогательных библиотек
Демонстрация на экране